Lecture 22 — LCA, Binary Lifting & Euler Tour

> Year Curriculum · Prerequisites: Week 16 (BFS/DFS), Week 20 (trees: depth, subtree sizes), Week 9 (bit manipulation) · Estimated time: 2–2.5 hours

1. Overview

Many tree problems ask the same few questions over and over: *how far apart are two nodes?*, *who is the k-th ancestor of v?*, *what is the sum of the values in v's subtree?* With up to 2·10^5 nodes and 2·10^5 queries, walking the tree for every query costs O(n) each and is far too slow. This lecture covers two preprocessing ideas that make these queries fast:

1. **Binary lifting** stores, for every node, its ancestors at distances 1, 2, 4, 8, …. Any upward walk of length k is then a sequence of at most log2(k) jumps. On top of it we build the **lowest common ancestor** (LCA) query in O(log n).
2. **Euler tour / tree flattening** numbers the nodes in DFS order so that every subtree becomes a contiguous segment of an array. Subtree questions then become array range questions, which we will answer with the segment trees and Fenwick trees of Weeks 23–25.

Both ideas are used constantly: LCA is the backbone of path queries, distance queries and "difference on tree" tricks; tree flattening is the bridge between tree problems and range data structures. Later, heavy-light decomposition (Nationals track) generalizes both.

2. Lowest common ancestor

Root the tree at some node (we use node 1) and compute `depth[v]`, the number of edges from the root to `v`. Node `u` is an **ancestor** of `v` if `u` lies on the path from `v` to the root; by convention every node is an ancestor of itself. The **lowest common ancestor** LCA(a, b) is the common ancestor of `a` and `b` with the largest depth.

The unique path between `a` and `b` goes up from `a` to LCA(a, b), then down to `b`. Hence

```
dist(a, b) = depth[a] + depth[b] - 2 * depth[LCA(a, b)]
```

and the same decomposition works for sums of node values along the path (see Section 6).

A naive LCA first lifts the deeper node one step at a time until depths match, then lifts both nodes together until they meet. That is O(depth) per query, which is Θ(n) on a path-shaped tree. Binary lifting replaces single steps with power-of-two jumps.

3. Binary lifting

3.1 The table

Define `up[j][v]` as the ancestor of `v` that is exactly 2^j levels higher. If that would go above the root we clamp it to the root (or, alternatively, to a sentinel node 0 meaning "does not exist"). The recurrence is the whole idea:

```
up[0][v] = parent(v)
up[j][v] = up[j-1][ up[j-1][v] ]
```

because jumping 2^j levels is jumping 2^(j−1) levels twice. With `LOG` levels such that 2^LOG exceeds the maximum depth (LOG = 18 suffices for n ≤ 2·10^5), the table has n·LOG entries and is filled in O(n log n).

The order of filling matters. `up[j][v]` reads `up[j-1][w]` for some other node `w`, so the **entire** level j−1 must be complete before level j is computed. The simplest way is the double loop "for j, for v". (Filling "for v in DFS order, for j" also works, because all ancestors of `v` are processed before `v`.)

Example on the path 1–2–3–4–5–6 rooted at 1:

v	1	2	3	4	5	6
up[0][v]	1	1	2	3	4	5
up[1][v]	1	1	1	2	3	4
up[2][v]	1	1	1	1	1	2

3.2 k-th ancestor

To climb k levels, write k in binary and perform one jump per set bit: for k = 13 = 8 + 4 + 1 we apply `up[3]`, `up[2]` and `up[0]` in any order (the jumps commute because they all go straight up). That is O(log n) per query.

CSES 1687 *Company Queries I* asks for the k-th ancestor or −1 if it does not exist. Here the sentinel version is convenient: let node 0 be "nobody", with `up[j][0] = 0` and the root's parent equal to 0. Then any jump past the root lands in 0 and stays there.

3.3 LCA query

Given `a` and `b`:

1. Assume `depth[a] ≥ depth[b]` (swap otherwise) and lift `a` by `depth[a] − depth[b]` levels. Now both nodes have the same depth.
2. If `a == b`, then `b` was an ancestor of the original `a`; return it.
3. Otherwise, for `j` from `LOG−1` down to 0: if `up[j][a] != up[j][b]`, set `a = up[j][a]`, `b = up[j][b]`.
4. Return `up[0][a]`.

Why it works. Let `d` be the depth of the LCA. Throughout step 3, `a` and `b` are distinct nodes at the same depth, both strictly below the LCA. A jump of 2^j that lands on two different nodes stays strictly below the LCA, so it is safe. A jump that lands on the same node lands on the LCA or above it; we cannot tell which, so we refuse it. Trying jumps from large to small is exactly binary search on "how many levels can I climb while staying below the LCA": after the loop, `a` and `b` are the two children of the LCA on the respective paths, and their common parent is the answer.

Worked example. In the tree with edges 1–2, 1–3, 3–4, 3–5 (rooted at 1), query LCA(4, 2). Depths are 0, 1, 1, 2, 2 for nodes 1..5. Lift 4 by 2−1 = 1 level to get 3. Now `a = 3`, `b = 2`, both at depth 1 and different. Every jump from 3 or 2 lands at the root 1, so all jumps are refused, and the answer is `up[0][3] = 1`. The distance is 2 + 1 − 2·0 = 3, matching the path 4–3–1–2.

3.4 Carrying data along the jumps

The same doubling works for aggregates. If edges have weights, define `mx[j][v]` as the maximum edge weight among the 2^j edges directly above `v`:

```
mx[0][v] = weight(v, parent(v))
mx[j][v] = max(mx[j-1][v], mx[j-1][ up[j-1][v] ])
```

When answering a path query, every time you move `a` or `b` by a jump, also fold the corresponding `mx` value into the answer. Any associative operation works: sum, min, max, gcd, even "composition of functions". A famous application is Codeforces 609E: build an MST (Week 19), and for each non-tree edge (u, v, w) the best spanning tree containing it costs `MST − maxEdgeOnPath(u, v) + w`.

3.5 Complexity

Preprocessing is O(n log n) time and memory; each k-th ancestor or LCA query is O(log n). For n = q = 2·10^5 this is roughly 7·10^6 operations — trivial.

Memory layout tip: `up[j][v]` (level-major) is cache-friendlier for the build loop, while `up[v][j]` keeps one node's jumps together. Both are fine at these sizes.

4. Avoiding stack overflow

A recursive DFS on a path of 2·10^5 nodes uses 2·10^5 stack frames. On Codeforces (256 MB stack) this is fine, but on many judges (including some CSES setups and most local machines with an 8 MB default stack) it can crash. For computing parents and depths, a BFS is enough: process nodes in BFS order, and each node's parent is discovered before the node itself. For the Euler tour we need a genuine DFS order; Section 7 shows an iterative DFS with an explicit stack and a per-node edge iterator.

5. Euler tour and tree flattening

Run a DFS and assign each node an **entry time** `tin[v]` (1, 2, 3, … in the order nodes are first visited). Let `tout[v]` be the largest entry time inside the subtree of `v` — equivalently the value of the timer when the DFS leaves `v`. Because DFS visits an entire subtree before leaving it, the nodes of the subtree of `v` are exactly those with entry times in `[tin[v], tout[v]]`.

```
        1          visit order: 1 2 3 4 5
       / \         tin : 1→1, 2→2, 3→3, 4→4, 5→5
      2   3        tout: 1→5, 2→2, 3→5, 4→4, 5→5
         / \
        4   5
```

Two immediate consequences:

- **Ancestor test in O(1):** `u` is an ancestor of `v` iff `tin[u] ≤ tin[v] ≤ tout[u]`.
- **Subtree ↔ range:** put `value[v]` at index `tin[v]` of an array `A`. Then "sum/min/max over the subtree of `v`" is "sum/min/max over `A[tin[v] .. tout[v]]`". Point updates on a node are point updates on `A`.

There is a second kind of tour, the **full Euler tour**, which writes a node every time the DFS passes through it (on entry and after returning from each child). It has length 2n − 1. For the tree above it is `1 2 1 3 4 3 5 3 1`. Between the first occurrences of `a` and `b` the tour visits exactly the path between them plus some subtrees hanging below it, and the shallowest node in that window is LCA(a, b). With a sparse table (Week 24) over depths, this gives O(1) LCA queries after O(n log n) preprocessing — useful when q is in the millions.

6. Classic reductions

Subtree sums with point updates (CSES 1137). Fenwick tree over `A`. Update: add `x − value[s]` at `tin[s]`. Query: range sum `[tin[s], tout[s]]`. O(log n) each.

Root-to-node path sums with point updates (CSES 1138). Let `f(s)` be the sum of values on the path from the root to `s`. Node `v` contributes to `f(s)` exactly when `s` is in the subtree of `v`. So changing `value[v]` by `d` means "add `d` to `f` on the range `[tin[v], tout[v]]`", and reading `f(s)` is a point query at `tin[s]`. A Fenwick tree used as a difference array does both in O(log n).

General path sums. With `f` as above and `c = LCA(a, b)`: `sum(a..b) = f(a) + f(b) − 2·f(c) + value[c]`.

Counting paths through each node (CSES 1136). For each path (a, b), do `cnt[a]++`, `cnt[b]++`, `cnt[c]--`, `cnt[parent(c)]--`, then take subtree sums. This "difference on a tree" trick needs only the LCA.

7. Reference implementations

7.1 LCA and distances (CSES 1135 *Distance Queries*)

Iterative BFS to get parents and depths, then the level-by-level table. Tested on the CSES sample and against a brute-force BFS on 300 random trees.

```cpp
#include <bits/stdc++.h>
using namespace std;

// CSES 1135 - Distance Queries: LCA with binary lifting.
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < n - 1; i++) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    int LOG = 1;
    while ((1 << LOG) <= n) LOG++;
    vector<vector<int>> up(LOG, vector<int>(n + 1));
    vector<int> depth(n + 1, 0);

    // Iterative BFS from root 1 (avoids deep recursion on path-like trees).
```

```
    vector<int> order;
    order.reserve(n);
    vector<bool> seen(n + 1, false);
    order.push_back(1);
    seen[1] = true;
    up[0][1] = 1;                       // root's parent is itself
    for (size_t i = 0; i < order.size(); i++) {
        int v = order[i];
        for (int u : adj[v]) {
            if (!seen[u]) {
                seen[u] = true;
                up[0][u] = v;
                depth[u] = depth[v] + 1;
                order.push_back(u);
            }
        }
    }
    for (int j = 1; j < LOG; j++)
        for (int v = 1; v <= n; v++)
            up[j][v] = up[j - 1][up[j - 1][v]];

    auto lift = [&](int v, int k) {     // k-th ancestor (clamped at root)
        for (int j = 0; j < LOG; j++)
            if (k >> j & 1) v = up[j][v];
        return v;
    };
    auto lca = [&](int a, int b) {
        if (depth[a] < depth[b]) swap(a, b);
        a = lift(a, depth[a] - depth[b]);
        if (a == b) return a;
        for (int j = LOG - 1; j >= 0; j--)
            if (up[j][a] != up[j][b]) {
                a = up[j][a];
                b = up[j][b];
            }
        return up[0][a];
    };

    while (q--) {
        int a, b;
        cin >> a >> b;
        int c = lca(a, b);
        cout << depth[a] + depth[b] - 2 * depth[c] << '\n';
    }
}
```

7.2 k-th ancestor with a sentinel (CSES 1687 *Company Queries I*)

```
#include <bits/stdc++.h>
using namespace std;

// CSES 1687 - Company Queries I: k-th ancestor with binary lifting.
// Employees 2..n are given their boss; node 0 is a sentinel "nobody".
```

```cpp
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    const int LOG = 18;                  // 2^18 > 2*10^5
    vector<array<int, LOG>> up(n + 1);
    up[0].fill(0);
    up[1].fill(0);
    for (int v = 2; v <= n; v++) cin >> up[v][0];
    // Fill level by level: level j needs all of level j-1 to be complete.
    for (int j = 1; j < LOG; j++)
        for (int v = 1; v <= n; v++)
            up[v][j] = up[up[v][j - 1]][j - 1];
    while (q--) {
        int x, k;
        cin >> x >> k;
        for (int j = 0; j < LOG && x != 0; j++)
            if (k >> j & 1) x = up[x][j];
        cout << (x == 0 ? -1 : x) << '\n';
    }
}
```

7.3 Euler tour + Fenwick tree (CSES 1137 *Subtree Queries*)

The DFS is iterative: `it[v]` remembers which neighbour of `v` to try next, so each edge is examined once.

```cpp
#include <bits/stdc++.h>
using namespace std;

// CSES 1137 - Subtree Queries: Euler tour (tin/tout) + Fenwick tree.
struct Fenwick {
    int n;
    vector<long long> t;
    Fenwick(int n) : n(n), t(n + 1, 0) {}
    void add(int i, long long d) { for (; i <= n; i += i & -i) t[i] += d; }
    long long sum(int i) const { long long s = 0; for (; i > 0; i -= i & -i) s += t[i];
return s; }
    long long sum(int l, int r) const { return sum(r) - sum(l - 1); }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> val(n + 1);
    for (int i = 1; i <= n; i++) cin >> val[i];
    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < n - 1; i++) {
        int a, b;
```

```
        cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    // Iterative DFS computing entry time tin[v] and exit time tout[v]
    // (tout = largest tin inside the subtree of v).
    vector<int> tin(n + 1), tout(n + 1), parent(n + 1, 0);
    vector<size_t> it(n + 1, 0);
    int timer = 0;
    vector<int> st = {1};
    tin[1] = ++timer;
    while (!st.empty()) {
        int v = st.back();
        if (it[v] < adj[v].size()) {
            int u = adj[v][it[v]++];
            if (u == parent[v]) continue;
            parent[u] = v;
            tin[u] = ++timer;
            st.push_back(u);
        } else {
            tout[v] = timer;
            st.pop_back();
        }
    }
    Fenwick fw(n);
    for (int v = 1; v <= n; v++) fw.add(tin[v], val[v]);
    while (q--) {
        int type;
        cin >> type;
        if (type == 1) {
            int s; long long x;
            cin >> s >> x;
            fw.add(tin[s], x - val[s]);
            val[s] = x;
        } else {
            int s;
            cin >> s;
            cout << fw.sum(tin[s], tout[s]) << '\n';
        }
    }
}
```

8. Common mistakes

- **LOG too small.** You need 2^LOG > maximum depth. For n ≤ 2·10^5 use 18; computing it from n (as in 7.1) avoids the issue.
- **Wrong fill order.** Computing `up[j][v]` before `up[j-1]` is complete for all nodes produces silently wrong ancestors when node ids are not in topological order.
- **Uninitialized root parent.** If `up[0][root]` is garbage, jumps index out of bounds. Use the root itself (clamping) or a sentinel 0 with `up[j][0] = 0`.

- **Forgetting to equalize depths**, or returning `a` instead of `up[0][a]` at the end of the LCA loop.
- **Jumping when ancestors are equal.** The condition is `up[j][a] != up[j][b]`; inverting it overshoots.
- **Recursive DFS on deep trees** → stack overflow on judges with small stacks. Use BFS order or an explicit stack.
- **Euler tour off-by-one.** `tout[v]` must equal the largest `tin` in the subtree. If you increment the timer on exit too (a "2n tour"), subtree ranges change meaning — be consistent.
- **`int` overflow** in subtree sums: values up to 10^9 times 2·10^5 nodes needs `long long`.

9. Practice problems

Warm-up:

- CSES 1687 — *Company Queries I* — https://cses.fi/problemset/task/1687
- CSES 1688 — *Company Queries II* — https://cses.fi/problemset/task/1688
- CSES 1135 — *Distance Queries* — https://cses.fi/problemset/task/1135
- CSES 1137 — *Subtree Queries* — https://cses.fi/problemset/task/1137

Core:

- CSES 1138 — *Path Queries* — https://cses.fi/problemset/task/1138
- CSES 1136 — *Counting Paths* — https://cses.fi/problemset/task/1136
- Codeforces 1328E — *Tree Queries* — https://codeforces.com/problemset/problem/1328/E
- Codeforces 191C — *Fools and Roads* — https://codeforces.com/problemset/problem/191/C

Challenge:

- Codeforces 609E — *Minimum spanning tree for each edge* — https://codeforces.com/problemset/problem/609/E
- Codeforces 519E — *A and B and Lecture Rooms* — https://codeforces.com/problemset/problem/519/E
- Codeforces 620E — *New Year Tree* (Euler tour + lazy segment tree, after Week 25) — https://codeforces.com/problemset/problem/620/E
- Codeforces 343D — *Water Tree* — https://codeforces.com/problemset/problem/343/D

10. Further reading

- CP-Algorithms: "Lowest Common Ancestor — Binary Lifting" and "LCA with Euler tour + RMQ" — https://cp-algorithms.com/graph/lca_binary_lifting.html
- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 18 "Tree queries" (free PDF at https://cses.fi/book/book.pdf).

- USACO Guide: "Binary Jumping" and "Euler Tour Technique" modules — https://usaco.guide/plat/binary-jump and https://usaco.guide/gold/tree-euler
- Looking ahead: Week 23 (segment trees) and Week 24 (Fenwick trees, sparse tables) supply the range structures used on flattened trees; the Nationals track covers heavy-light decomposition, which extends these ideas to arbitrary path updates.